RF Predator
Production and Operator Guide

Raspberry Pi 5 + HackRF One + Phone Web GUI
Single-source production, implementation, and operator workflow manual

Platform	Raspberry Pi 5 tactical SDR appliance
Primary sensor	HackRF One, with optional GPS and future DF-capable inputs
Operator interface	Phone-sized browser UI served locally from the Pi
Purpose of this guide	Provide one complete guide for production, project creation, and day-to-day use

This guide combines the project definition, dependency manifest, service topology, UI specification, API contracts, function requirements, production milestones, and operational how-to workflows into a single handoff document.

Table of Contents

Update the field in Word if a live table of contents is needed.

1. Executive summary

RF Predator is a Raspberry Pi 5 tactical SDR appliance that runs all RF processing, mission logic, storage, and integrations on the Pi while exposing a phone-sized web GUI as the operator control head. The design preserves the useful mission-driven ideas from the source workflow: persistent mission status, mission modes, mission configuration, spectrum-first interaction, hits and events analysis, network filtering, map/DF views, and FHOP scaffolding.

At-a-glance summary

Primary compute node	Raspberry Pi 5
Primary SDR	HackRF One
Control/display head	Phone browser connected to the Pi
Runtime model	systemd-enabled backend, worker, watchdog, and web stack
Primary missions	Simple mission, DF/maps module, FHOP mission scaffold
Safety boundary	Receive, analyze, log, map, export, and integrate only; no offensive transmit or jamming features

2. Scope, safety, and design intent

This document is intended to help build the project and also explain how the finished system should be used. It deliberately keeps the project within a lawful receive/analyze/log/map workflow.

- Included: receive, analyze, classify, search, scan, log, play back audio, export sessions, plot detections, future DF-capable architecture, and ATAK/CoT-style integration.
- Excluded: jamming, active effects, coercive radio functions, offensive protocol abuse, or any feature intended to impair external systems.
- Production goal: a Pi 5 appliance that boots, self-starts, presents a stable phone-sized interface, and remains usable over long sessions.

3. System overview

Pi role	Runs all services, controls the SDR, stores data, hosts the API and web GUI, and performs

	export and integration actions.
Phone role	Displays the tactical web UI, sends commands, views maps, events, and spectrum, and optionally plays streamed or on-demand audio.
Connectivity model	Phone connects to the Pi over a local path such as Pi-hosted Wi-Fi AP, LAN, or a field network.
Production objective	A bootable appliance that keeps operating even if the phone disconnects and reconnects later.

4. Architecture layers

Platform layer

Raspberry Pi OS or Ubuntu Server, systemd, journald, nginx, local filesystem, USB stack, optional GPS stack.

Hardware layer

HackRF access, USB lifecycle management, optional GPS and heading sources, optional future DF input layer.

Worker layer

IQ intake, FFT, waterfall, thresholding, marker management, audio demod/capture, clustering, correlation, and event generation.

Mission layer

Mission state machine, Simple mission modes, FHOP mission support, DF/maps module, dwell policies, and target/exclude logic.

Storage layer

SQLite plus file-backed audio, exports, session archives, maps, waveforms, and overlays.

API layer

REST endpoints for command/config and WebSocket channels for live spectrum, health, events, and alerts.

Frontend layer

Phone-first web GUI with a fixed mission shell, spectrum-first page design, and compact tactical operator views.

Integration layer

CoT/ATAK publishing, future external consumers, client audio support, and export pathways.

5. Service topology and systemd design

Core services

Service	Purpose	Starts at boot	Critical notes
rfpredator-backend.service	REST API, WebSocket manager, mission orchestration, UI command intake	Yes	UI should still load even if the SDR worker is degraded.
rfpredator-worker.service	HackRF control, SDR processing, FFT/waterfall, markers, hits/events, audio capture	Yes	Must release HackRF cleanly during stop or restart.
rfpredator-watchdog.service	Health polling, stale stream detection, recovery logic	Yes	Restarts must be deliberate and bounded.
rfpredator-maptiles.service	Offline map tiles and overlay serving	Optional	Keep this independent from SDR load.
rfpredator.target	Groups the appliance services for enable/start behavior	Yes	The system should be enabled through this target.

- All required services auto-start at boot and restart safely on failure.
- The frontend remains reachable even when the HackRF is unplugged or the worker is restarting.
- The worker heartbeat is visible to the backend and watchdog.
- The backend never directly blocks on heavy SDR work; the worker owns the SDR hot path.

6. Repository and file layout

Recommended top-level structure:

```
rf-predator/
  docs/            # architecture, product, API, workflows
  backend/         # FastAPI app, services, worker coordination, models, repos
  frontend/        # phone-first web app
  native/          # HackRF and FFT helpers
  deploy/          # systemd units, nginx config, install/update scripts
```

```
data/            # sessions, audio, exports, maps, waveforms, logs
tools/           # helper prompts, sample captures, validation tools
```

7. Dependency manifest

Operating system packages

Package group	Examples	Required now	Notes
Core runtime	python3, python3-venv, python3-dev, build-essential, git, curl, cmake, pkg-config	Yes	Needed for backend and native helpers.
Web and storage	sqlite3, nginx, jq	Yes	Forms the web stack and local persistence layer.
SDR and native	libusb-1.0-0-dev, hackrf, libhackrf-dev, libfftw3-dev	Yes	Core RF and FFT toolchain.
Audio tools	ffmpeg, sox	Yes	Audio conversion and clip handling.
Ops support	rsync, unzip, logrotate	Yes	Backups, archives, and operational maintenance.
Optional GPS/geospatial	gpsd, gpsd-clients, gdal-bin, libgdal-dev, proj-bin, libproj-dev	Optional	Enable only when those feature paths are needed.

Python dependencies

Area	Libraries	Required now	Notes
API and config	fastapi, uvicorn[standard], pydantic, pydantic-settings, python-multipart	Yes	Core service and configuration stack.
Realtime	websockets, anyio,	Yes	Required for

	httpx		streaming state and live UI updates.
Persistence	sqlalchemy, alembic, aiosqlite	Yes	SQLite first, migration-capable design.
Logging and data	PyYAML, structlog, orjson	Yes	Fast structured logging and config parsing.
DSP	numpy, scipy	Yes	Primary numeric processing path unless native helpers own more of the hot path.
Audio	soundfile	Yes	Clip file handling. Add pydub only if needed later.
Optional geospatial	shapely, pyproj, rasterio	Optional	Bring in only when map/overlay features need them.

Frontend dependencies

Area	Libraries	Required now	Notes
Core app	react, react-dom, typescript, vite	Yes	Fast phone-first web build.
Routing and state	react-router-dom, zustand, zod	Yes	Simple route model and predictable local state.
Styling	tailwindcss, postcss, autoprefixer, clsx	Yes	Controlled mobile styling system.
Maps	leaflet or maplibre-gl	Optional early	Use one, not both, in the first build.
Testing	vitest, @testing-library/react,	Yes	Unit and end-to-end

	playwright	checks.

8. Filesystem and storage model

Configuration	/etc/rfpredator/config.yaml and profile files under /etc/rfpredator/profiles/
Runtime state	/var/lib/rfpredator/db, sessions, audio, exports, waveforms, maps, overlays
Logs	journald plus /var/log/rfpredator/
Ownership	Runtime directories owned by the rfpredator service user
Persistence rule	Database holds structured records; audio is file-backed; exports are deterministic archives.

9. Core data model

Entity	Key fields	Purpose	Production notes
Session	id, start/end, mission type, app version, notes	Container for one operator run	Should survive restarts and support export.
MissionState	mission type, mode, entered/exited times	Tracks mission transitions	Useful for debugging and operator audit.
Hit	freq, cluster key, hit count, RSSI, last seen, target state	Threshold crossing record	One of the core operator views.
Event	times, marker, protocol, source/dest metadata, viewed flag, audio link	Processed activity tied to a marker or decoder result	Must support audio playback and export.
Marker	freq, bandwidth, source type, state, audio/DF flags	Logical channel resource	Preserve the operator mental model.

Search/Target/Exclude	freq ranges, bandwidth, enabled, priority/source	Mission control lists	Must be importable/exportable.
DfLob	bearing values, quality, emitter key, timestamp	Direction finding record	Only valid when DF capability is enabled.
Waveform	definition JSON, built-in flag, enabled	FHOP support object	Needed early so FHOP scaffolding has persistence.
Alert	source, level, message, viewed state	Operator-visible notices	Support system alerts and workflow alerts.

10. API contract summary

REST endpoints

Area	Representative endpoints	Purpose	Notes
System	GET /api/system/health, GET /api/system/info	Health, versioning, runtime summary	Must remain available even when the worker is degraded.
Missions	GET /api/missions/current, POST /api/missions/set, POST /api/missions/simple/mode	Switch and inspect mission state	Mode changes must be atomic.
Spectrum and markers	POST /api/spectrum/cf, /gain, /threshold; marker CRUD routes	Core operator control path	These are among the most-used commands.
Search/Target/Exclude	CRUD routes plus dwell profile updates	Mission configuration	Should be safe to update without page reload.
Hits and events	GET /api/hits, GET	Analysis and playback	Filtering and sorting

	/api/hits/{id}/events, GET /api/events/{id}/audio		are crucial.
Network	GET /api/network/tree and node actions	Hierarchical decoder-driven analysis	Phone-friendly response shapes are preferred.
DF/Maps/FHOP	status, list, CRUD, and mode routes	Scaffold and advanced mission support	Capability-gated features must report clearly.
Exports and integrations	POST /api/export/session/{id}, integration tests	External interoperability and archive production	Avoid blocking the API during archive creation.

WebSocket channels

Channel	Carries	Required behavior	Notes
/ws/health	backend, worker, SDR, GPS, and storage health	Low-rate health updates	Used by the persistent shell.
/ws/spectrum	compact spectrum frames, threshold, markers	Bounded frame rate and browser-safe payloads	Do not send raw IQ to the UI.
/ws/waterfall	downsampled waterfall rows	Bounded memory and efficient painting	Separate from spectrum when helpful.
/ws/events	hit/event deltas	Fast card/list refresh without page reload	Support partial update model.
/ws/alerts	operator alerts and warnings	Visible badge or strip integration	Should remain concise and readable.

11. Phone web GUI specification

This sizing system is a derived engineering spec. It gives a coding helper explicit constraints for the phone-first web UI.

Viewport targets

Primary viewport target	390 x 844 CSS px portrait phone viewport
Secondary supported size	360 x 740 CSS px portrait
Primary layout rule	No horizontal scrolling on major pages
Scrolling rule	One primary scroll region per page at most; spectrum page should ideally have no page scroll.

Fixed shell sizing

Component	Target size	Padding / spacing	Notes
Top mission status bar	48 px height	8 px inline padding, 6 px item gap	Must stay fixed and always visible.
Right tab rail	44 px width	36 px button, 18 px icon, 8 px vertical gap	All primary pages stay reachable.
Bottom action strip	44 px height	8 px inline padding, 6 px button gap	Holds page-level actions.
Main content safe padding	N/A	6 px top/bottom, 8 px left/right	Never allow horizontal overflow.

Buttons and touch targets

Element	Spec	Typography	Production rule
Primary button	Min 40 px height, min 72 px width	13 px label, 8 px radius	Use labeled buttons for important actions.
Compact icon button	36 x 36 px	18 px icon	Good for tool rows and tab controls.
Minimum touch target	36 px absolute minimum	N/A	Prefer 40-44 px for primary controls.
Dangerous action control	Min 40 px height	13 px label	Never icon-only for destructive actions.

Typography and list sizing

Element	Size	Spacing	Notes
Status text	11-12 px	Compact gap model	For persistent shell metadata.
Body/action text	13 px	Standard control spacing	Primary readable size.
Hit card	Min 64 px height	8 px internal padding, 4 px inner gap	Should not feel cramped.
Event row	Min 52 px height	6 px x 8 px padding	Compact but readable.
Network accordion row	Min 44 px height	12 px child indent	Avoid desktop-style wide trees.
Mission drawer	Width min(92vw, 360px)	12 px padding and 12 px section gap	Phone-first configuration panel.

- Spectrum page: status bar fixed, spectrum dominant, tool row directly below spectrum, optional live audio in a collapsible or bounded panel.
- Hits and events pages: use stacked cards and compact list rows, not wide desktop tables.
- Network page: use accordion groups rather than a wide tree/table mash-up.
- DF page: show a clear disabled state when DF capability is unavailable.
- Map page: keep recenter and overlay toggles visible without forcing a long control scroll.

12. Mission model and operator modes

Simple mission modes

Mode	Purpose	Core behavior	Operator promise
Manual	Direct operator tuning and marker ownership	User places and controls markers directly, with explicit gain, threshold, and center-frequency control	Nothing surprises the operator.
Classify	Use idle resources without taking control away	Auto-assign idle markers to signals while preserving user	Background help without loss of user authority.

		markers	
Scan	Automated search/target workflow	Cycles search bands and targets with dwell logic and priority rules	Efficient wide-area coverage.
QuickScan	Single-marker rapid scan workflow	Fast step-through with dwell, delay, prev/next, and target-current actions	Minimal-friction tactical checking.

FHOP mission

The FHOP mission is planned as a later-phase module with waveform management, match and identify modes, emitter tracking, event filtering, and optional future DF integration. The architecture is prepared now so the project does not need a structural rewrite later.

13. Function requirements matrix

Function	What it must do	Dependencies	Production checks
connect_hackrf()	Open the device, verify state, publish connection status, prevent duplicate opens	libhackrf, libusb, native bridge	Handle missing device, access failure, and reconnect loops cleanly.
start_rx_stream()	Begin one active stream, allocate buffers, update heartbeat, publish worker state	native bridge, ring buffer, health state	Must not allow multiple active stream loops.
compute_fft_frame()	Window samples, compute FFT, normalize, package a browser-safe frame	numpy/scipy or FFTW helper	Latency must remain stable under sustained runtime.
detect_peaks()	Identify actionable peaks over threshold, merge adjacent bins	threshold engine, FFT frames	Avoid noise spike spam and duplicate marker placement.
add_marker()	Reserve a logical	marker store,	Marker state must be

	channel resource and tune it to the chosen frequency	allocator, worker state	visible to the UI immediately.
run_scan_cycle()	Execute search/target/exclude plan with dwell rules	mission controller, hardware manager, target lists	Accept pause/pass commands without state confusion.
record_hit()	Create or update clustered hit records	clustering engine, DB repo	Fast delta updates to the UI are required.
open_event_for_marker()	Create a live event, tie it to marker and metadata, optionally open audio clip	event service, storage service	Event lifecycle must survive marker reassignment gracefully.
apply_global_filter()	Implement OR within groups and AND across groups consistently	filter engine, view adapters	Must be deterministic and reusable across pages.
ingest_decoded_event()	Normalize protocol results into network/event structures	decoder adapters, repos	Should not break if decoder metadata is partial.
ingest_bearing_observation()	Persist and publish LOBs only when DF capability is valid	DF capability manager, DB repo	Never fake DF when no valid source exists.
export_session_archive()	Build deterministic session exports with DB records and audio	export service, storage repos	Should not freeze or block the main API path.

14. Production workflow for building the project

1. Stand up the base Pi image, install the required OS packages, create the service user, and prepare the filesystem layout under /etc, /var/lib, and /var/log.
2. Install the backend Python environment and verify that the API can start without the worker attached.
3. Build the frontend shell first so the phone UI is usable before SDR logic is introduced.
4. Implement the backend service and worker split before adding heavy SDR functions. Do not let the web server own the hot path.

5. Bring up HackRF control next: connect, disconnect, configure center frequency and gains, and start/stop RX safely before painting spectrum.
6. Implement spectrum and waterfall transport to the browser, then add marker overlay and manual controls.
7. Add the Simple mission state machine and then build Manual, Classify, Scan, and QuickScan one by one.
8. Add hits, events, playback, search/target/exclude/dwell persistence, and global filter behavior.
9. Add the decoder adapter framework and network view once the event backbone is stable.
10. Layer in maps, DF gating, exports, client audio, and ATAK/CoT integrations after the core mission loop is stable.
11. Add FHOP scaffolding only after the main Simple mission path is already operational end to end.

15. Operator how-to guide

15.1 Start-up workflow

12. Power the Pi 5 and verify that the appliance network path is available.
13. Connect the phone to the Pi-hosted or Pi-reachable network.
14. Open the RF Predator web URL from the phone browser.
15. Confirm the status bar shows backend healthy and worker healthy.
16. Connect the HackRF One if it is not already attached, then verify the hardware state becomes connected.
17. Open the system or health page if any service reports degraded before starting a mission.

15.2 Manual mode workflow

18. Set the mission to Simple and choose Manual mode.
19. Set or confirm center frequency, gain profile, threshold, and span or sample-rate behavior.
20. Use the spectrum page to locate peaks or frequencies of interest.
21. Place markers on frequencies you want to watch directly.
22. Enable live audio or event capture on selected markers as needed.
23. Rename or pin important frequencies when they need to remain recognizable later in the session.

15.3 Classify mode workflow

24. Switch to Classify when you want the system to use idle resources while preserving your own marker placements.
25. Keep user markers on the frequencies you care about most.
26. Allow the background logic to task unused markers to active signals.
27. Watch hit and event population increase while still retaining direct user control of your chosen markers.
28. If a background task becomes important, convert it into an explicit user marker or target.

15.4 Scan mode workflow

29. Open the mission drawer and define one or more search bands.

30. Add known targets and, if needed, excludes.
31. Choose a dwell profile that matches the search objective.
32. Start Scan mode and watch the current band, activity summary, and event creation flow.
33. Use pause or skip/pass actions if the current signal needs more or less attention.
34. Refine targets and excludes during the scan as you learn the local environment.

15.5 QuickScan workflow

35. Choose QuickScan when a rapid single-marker check is more useful than full scan or classify behavior.
36. Set dwell, delay, and duration.
37. Use previous and next actions to move through signals quickly.
38. Tag or target the current signal when it should be retained for later scan or event analysis.

15.6 Hits and events workflow

39. Open the Hits page to review clustered frequency activity.
40. Use All, Unknown, Target, or Exclude views to narrow what is on screen.
41. Select a hit to open its event history.
42. Play event audio from the event detail or playback drawer.
43. Mark viewed items as processed and use sort and filter controls to keep the list manageable.

15.7 Network workflow

44. Open the Network page after decoder-backed events exist.
45. Expand the network or node hierarchy using the accordion layout.
46. Rename important nodes when this helps operator memory.
47. Promote selected nodes to targets or place markers directly from the node actions.
48. Use this page to move from raw event traffic toward structured network understanding.

15.8 DF and map workflow

49. Open the DF page only when a valid DF capability is attached or enabled.
50. Use the compass rose and emitter list to watch live bearings and recent history.
51. Open the map page to see sensor position, detections, and valid LOB overlays.
52. Use recenter, label, and overlay toggles to simplify the display when the map becomes cluttered.

15.9 FHOP workflow

53. Open FHOP only after the main system backbone is proven stable.
54. Load built-in or custom waveforms.
55. Choose Match when you have known waveform candidates and Identify when surveying more broadly.
56. Review emitter and event cards and refine the waveform library over time.

15.10 Export and shutdown workflow

57. Export the current session before power-down when the run contains material worth preserving.

58. Confirm the export archive includes database records and any relevant audio clips.
59. Stop missions cleanly before full system shutdown so the HackRF is released correctly.
60. Use the backup script or archive plan at regular intervals for long-running deployments.

16. Troubleshooting workflow

Condition	Likely cause	Immediate action	Longer fix
UI loads but no spectrum	Worker stopped or HackRF not connected	Check health page and hardware state, then restart the worker if needed	Inspect worker logs and verify USB and HackRF package setup.
Spectrum visible but controls feel laggy	Too-high UI frame rate or browser load	Reduce spectrum update rate and verify bounded payload size	Profile the frontend canvas path and worker pacing.
Repeated worker restarts	HackRF release or reconnect bug or bad native state	Stop the stack, reconnect hardware, and start again cleanly	Harden device state transitions and stop paths.
Audio missing from events	Clip not opened or storage/audio pipeline issue	Check event metadata and clip file existence	Verify demod path and file-backed audio lifecycle.
Map blank or no overlays	Map tile service not available or no local tiles	Check map service health and data paths	Load local tiles and overlays and confirm route wiring.
DF page empty	No DF source or DF capability disabled	Confirm DF capability state	Do not expose or imply DF when unsupported.

17. Milestone plan

Milestone	Deliverables	Why it comes here	Done when
1. Shell and services	Backend skeleton, frontend shell, systemd units, nginx, health page	Establish appliance behavior first	Pi boots into a reachable UI and reports service health.

2. HackRF and spectrum	HackRF manager, worker, FFT, spectrum, waterfall, marker overlay	The system needs a live RF backbone before deeper mission logic	Operator can see and control live spectrum from the phone.
3. Simple mission core	Manual, Classify, Scan, QuickScan, search/target/exclude/dwell	This is the primary operating workflow	Mission state changes and operator actions work end to end.
4. Hits, events, and audio	Persistent hits and events, playback, global filter	Makes the system analytically useful	Operator can review, filter, and play session data.
5. Decoder and network	Decoder API scaffold and network view	Adds structured analysis	Node-based workflow is usable on a phone without desktop tables.
6. Maps and DF scaffold	Map service, DF gating, compass and map pages	Adds situational layout and future DF path	UI clearly distinguishes valid DF from unavailable DF.
7. Integrations	CoT/ATAK, client audio, export polish, watchdog hardening	Adds field interoperability	Exports and external publishing are stable.
8. FHOP scaffold	Waveform library, match and identify workflow, emitter cards	Advanced mission path last	No major architecture change is needed to add FHOP.

18. Coding-helper handoff notes

- Print the exact files to be created or modified before generating code.
- Keep backend and worker separate from the start.
- Do not couple the frontend directly to hardware access.
- Keep every milestone runnable on the Pi 5, even if features beyond the current milestone are stubbed.
- Follow the mobile sizing and no-horizontal-scroll rules exactly.
- Treat unsupported DF as explicitly unavailable, not silently inactive.
- Do not generate offensive transmit or jamming features.

19. Final production checklist

- Systemd target starts the appliance on boot.
- Backend and worker both report healthy state.
- Phone can reach the web UI reliably.
- HackRF connect/disconnect works repeatedly without stale state.
- Spectrum and markers are stable on a phone viewport.
- Simple mission modes work as designed.
- Hits, events, and audio playback persist across session use.
- The UI remains free of horizontal scrolling on major pages.
- Exports are deterministic and easy to retrieve.
- Safety boundary remains intact throughout implementation.